

Deep Learning with Pytorch



Artificial Intelligence / Machine Learning / Deep Learning



Artificial Neural Networks / Neuron/ Perceptron



Single-Layer Perceptron / Multi-Layer Perceptron



Deep Learning Networks



Deep Learning Frameworks

Overview of AI, Machine Learning, and Deep Learning

Artificial Intelligence (AI)

AI is the broad field focused on creating machines that can perform tasks that normally require human intelligence. These tasks include reasoning, problem-solving, understanding language, recognizing patterns, and decision-making.

→ *Goal*: Make computers think and act like humans.

Machine Learning (ML)

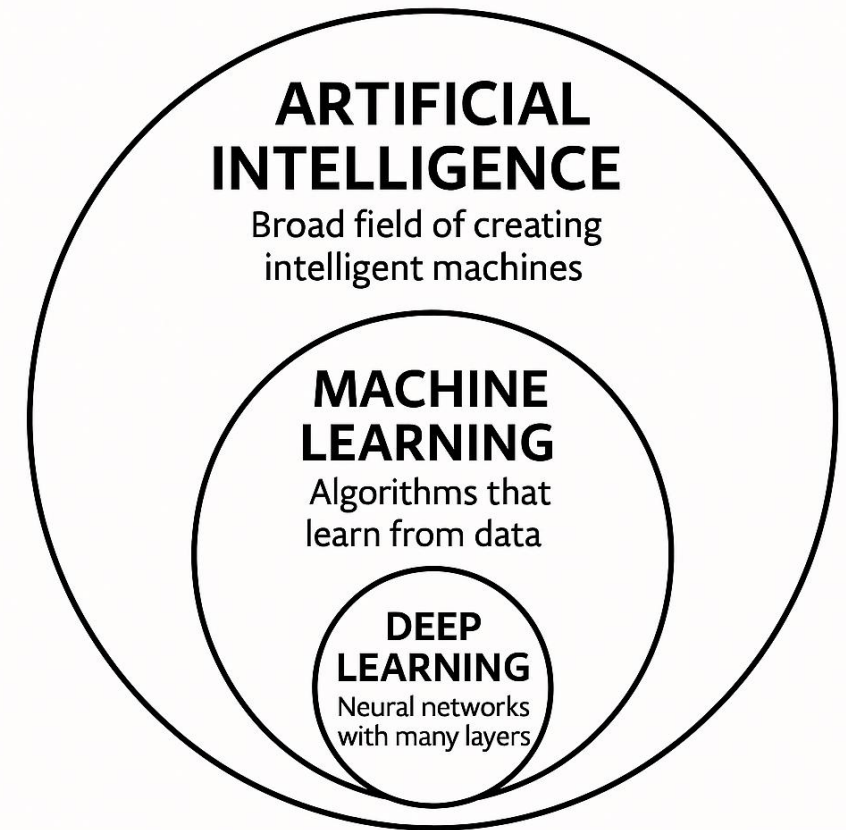
ML is a subset of AI. It allows machines to automatically learn from data without being explicitly programmed. Instead of hand-coding rules, the system *finds patterns* and *improves* its performance over time through experience.

→ *Goal*: Let machines learn from data.

Deep Learning (DL)

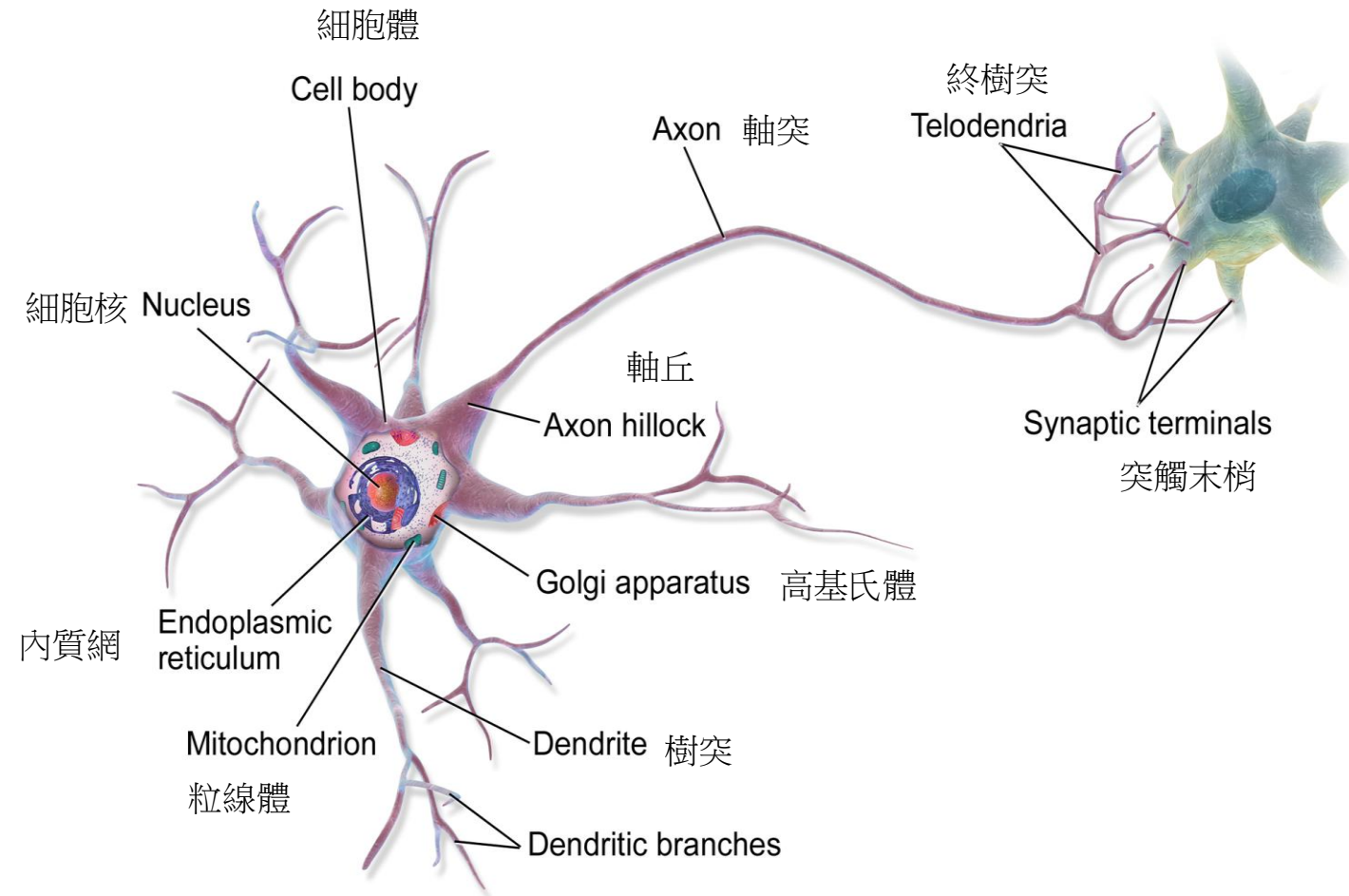
DL is a specialized branch of ML. It uses **neural networks** with many layers (hence "deep") to model complex patterns in large amounts of data. It's the key technology behind modern breakthroughs like image recognition, natural language processing, and autonomous driving.

→ *Goal*: Solve very complex tasks by using deep neural networks.



Artificial Neural Networks (ANNs)

Neuron 神經元

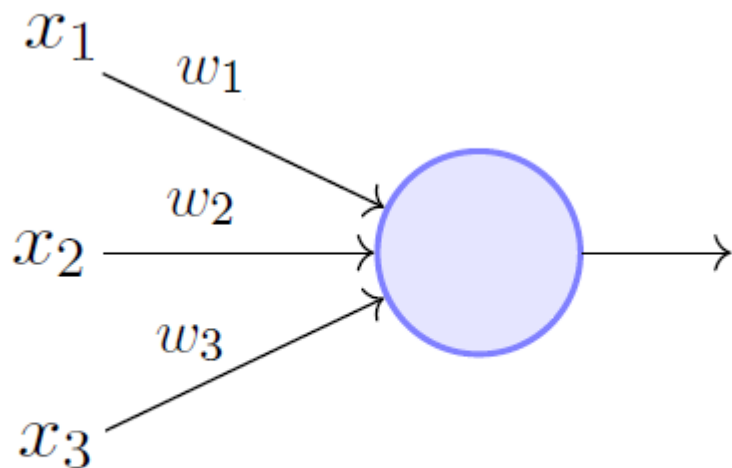


Perceptron

感知器

Perceptrons: An Introduction to Computational Geometry

感知器 (Perceptron) 是 [Frank Rosenblatt](#) 基於生物中神經細胞的概念，於 1957 年發明的「人造神經元」(Artificial Neuron)，是一種簡單的「二元線性分類器」。

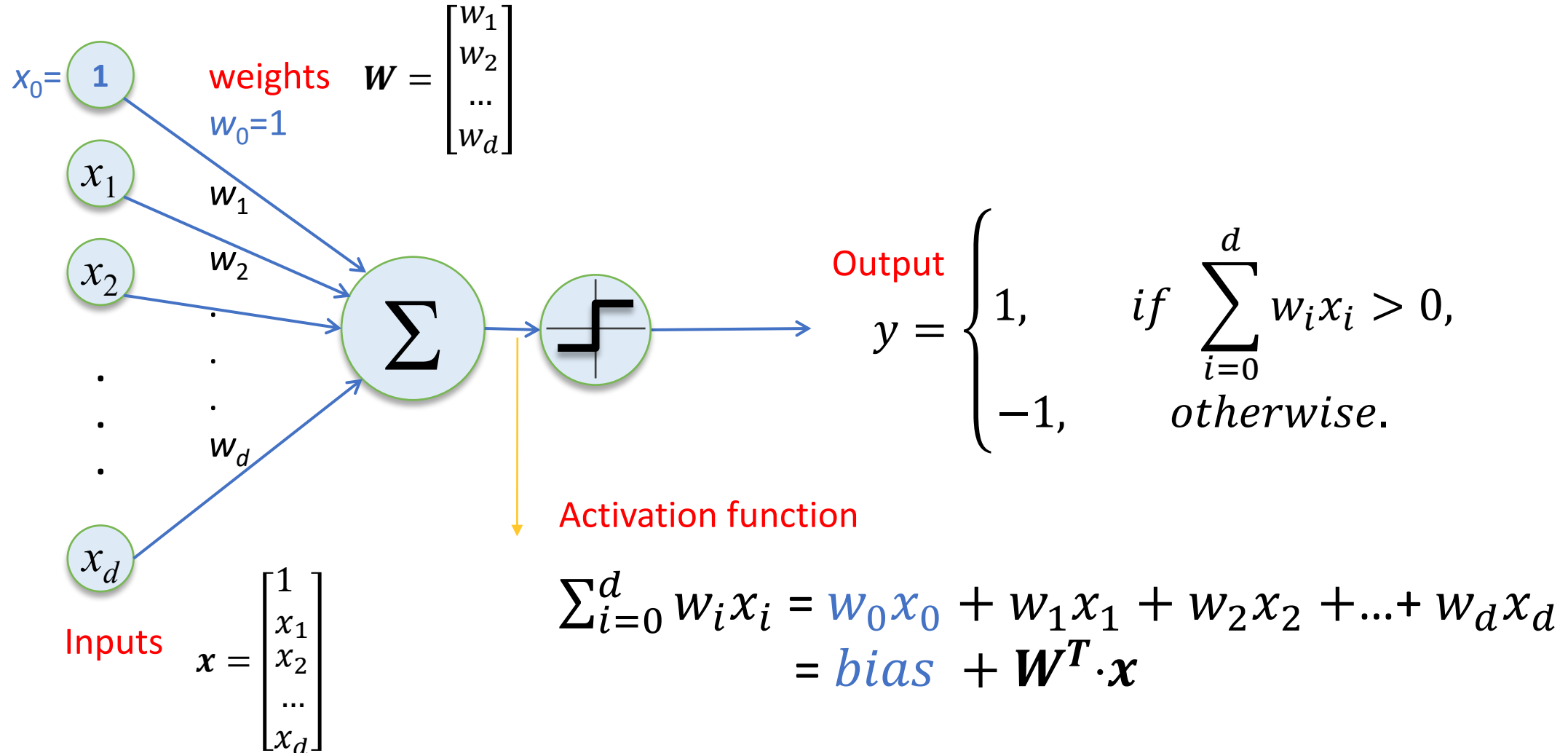


$$\text{output} = \begin{cases} 0 & \text{if } \sum_j w_j x_j \leq \text{threshold} \\ 1 & \text{if } \sum_j w_j x_j > \text{threshold} \end{cases}$$

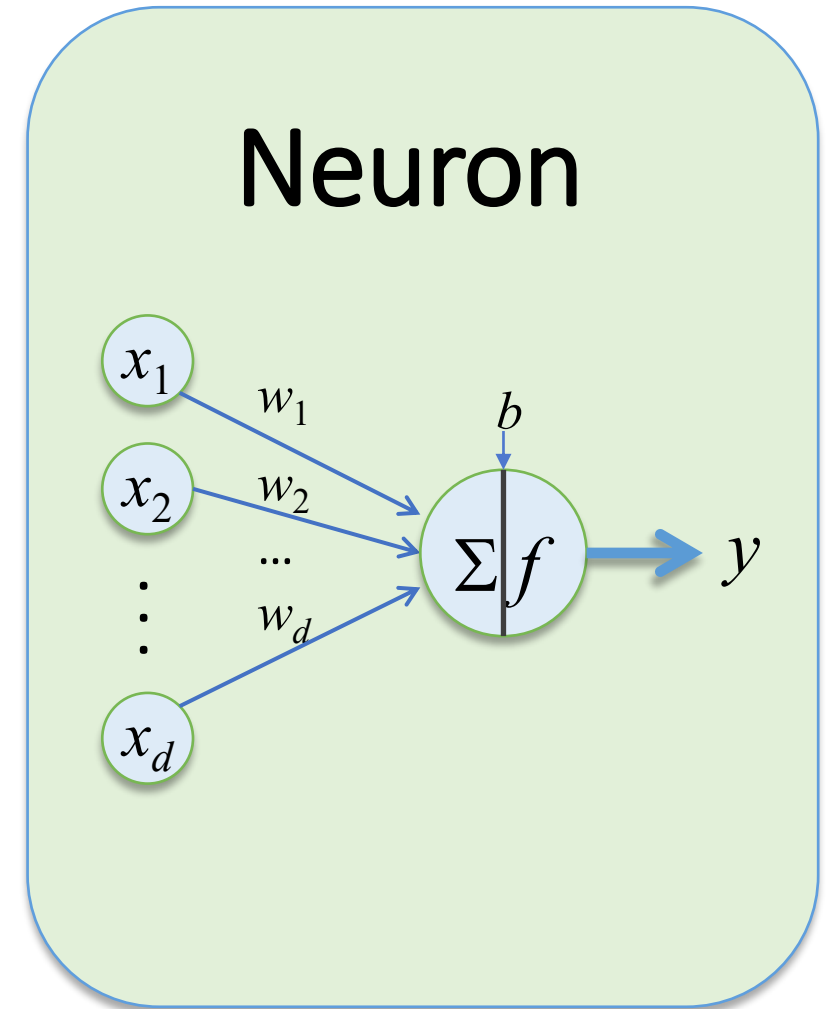
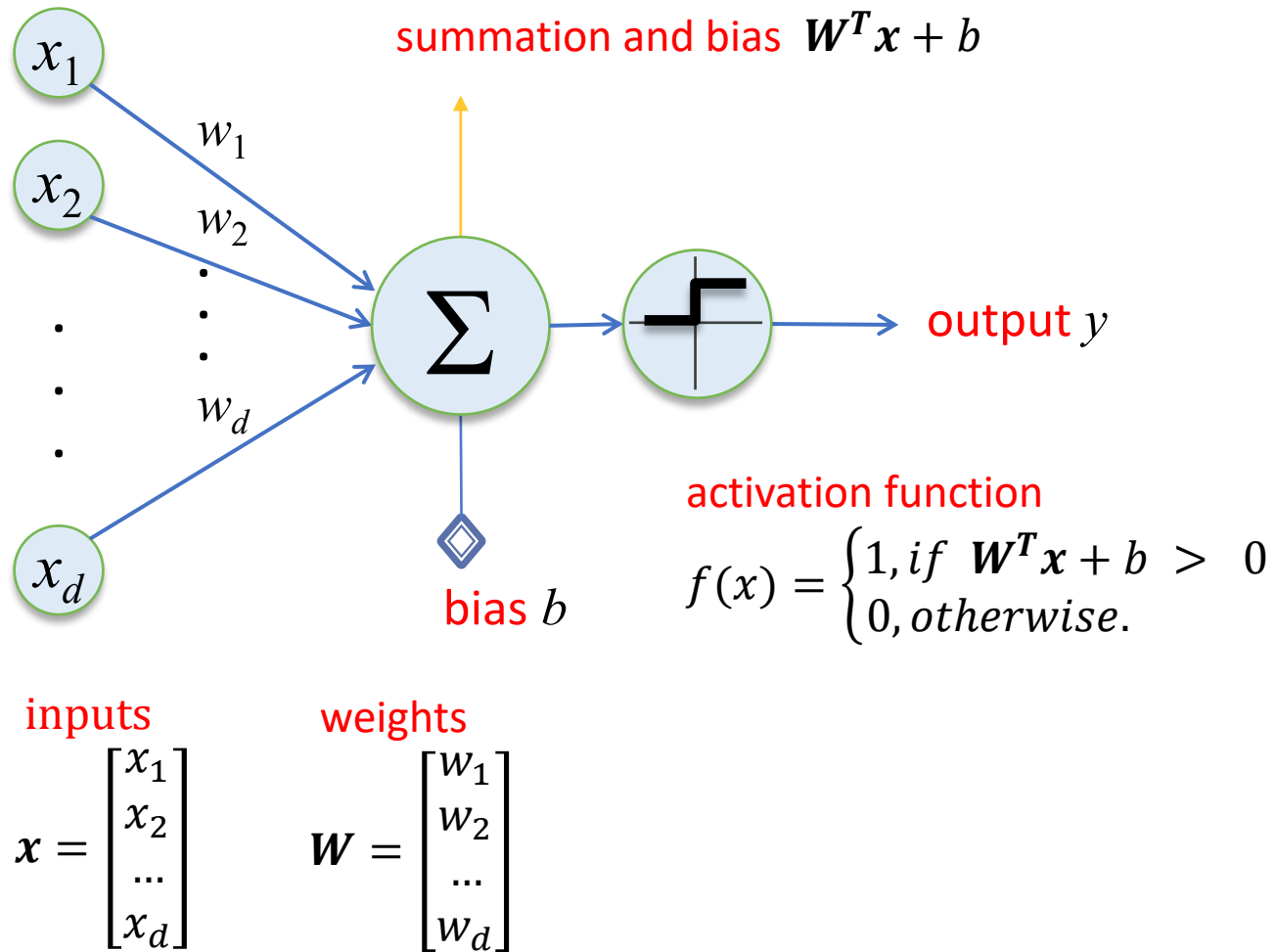
$$x_1, x_2, x_3, \text{output} \in \{0, 1\}$$

Perceptron Model (Minsky-Papert in 1969)

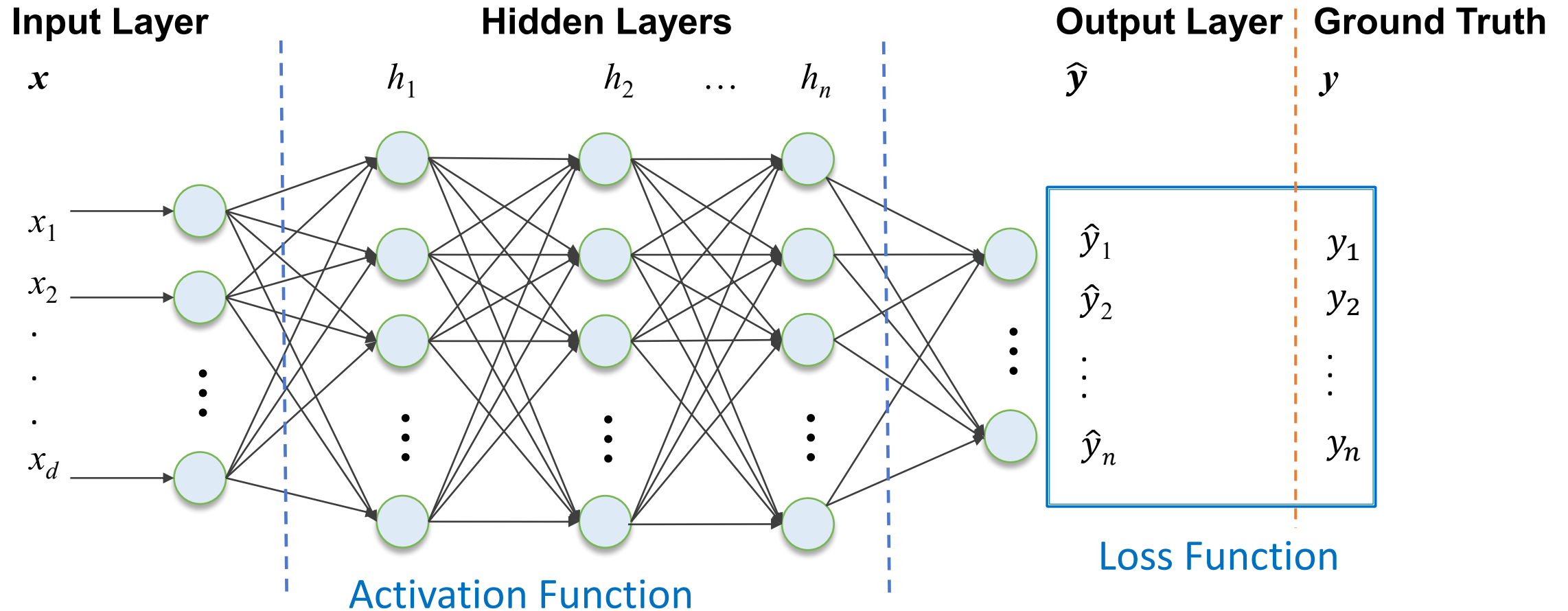
Perceptron — Fundamental Unit of a Neural Network



Single-Layer Perceptron



Multi-Layer Perceptron

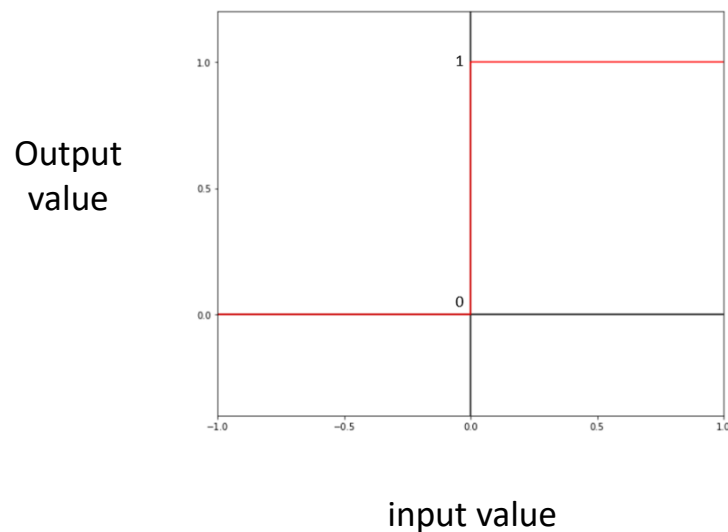


Activation Function

Activation functions that are functions of a single fold x from the previous layer or layers:

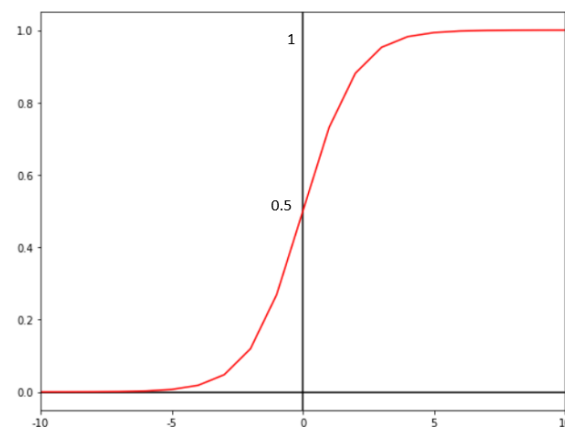
Binary Step function

$$f(x) = \begin{cases} 0 & \text{for } x \leq 0, \\ 1 & \text{for } x > 0. \end{cases}$$



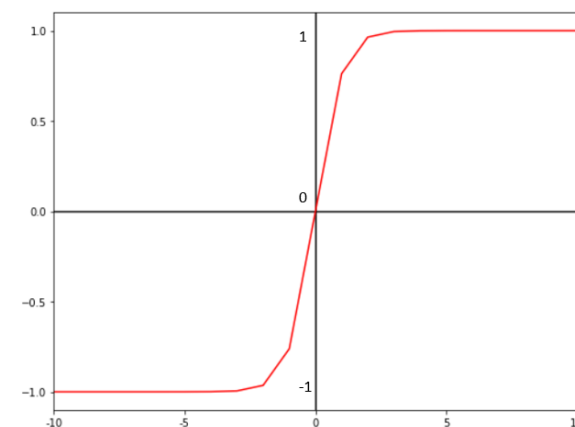
Sigmoid Function

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$



Tanh Function

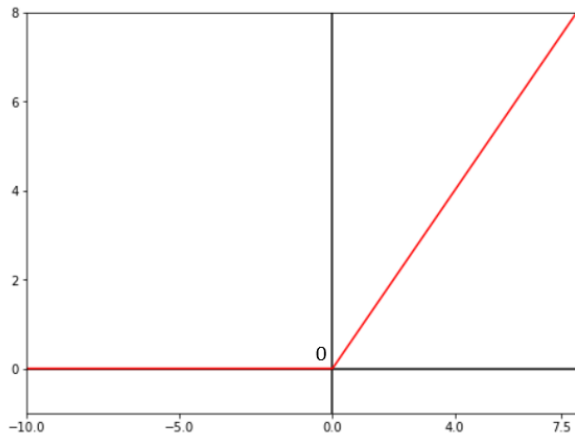
$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$



Non-linear Activation Functions

Rectified Linear Unit Function ReLU

$$f(x) = \begin{cases} 0 & \text{for } x \leq 0, \\ x & \text{for } x > 0. \end{cases}$$

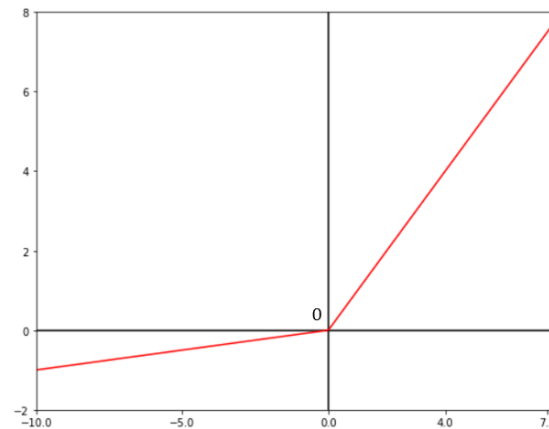


$$f(x) = \max(0, x)$$

Leaky ReLU Function

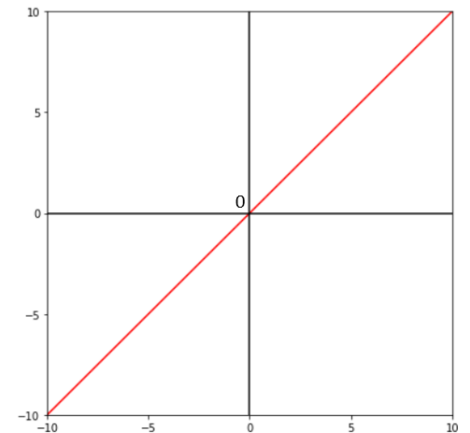
$$f(x) = \begin{cases} \alpha x & \text{for } x \leq 0, \\ x & \text{for } x > 0. \end{cases}$$

α : a small constant



Identity Function

$$f(x) = x$$



Activation functions that are not functions of a single fold x from the previous layer or layers:

Softmax Activation Function:

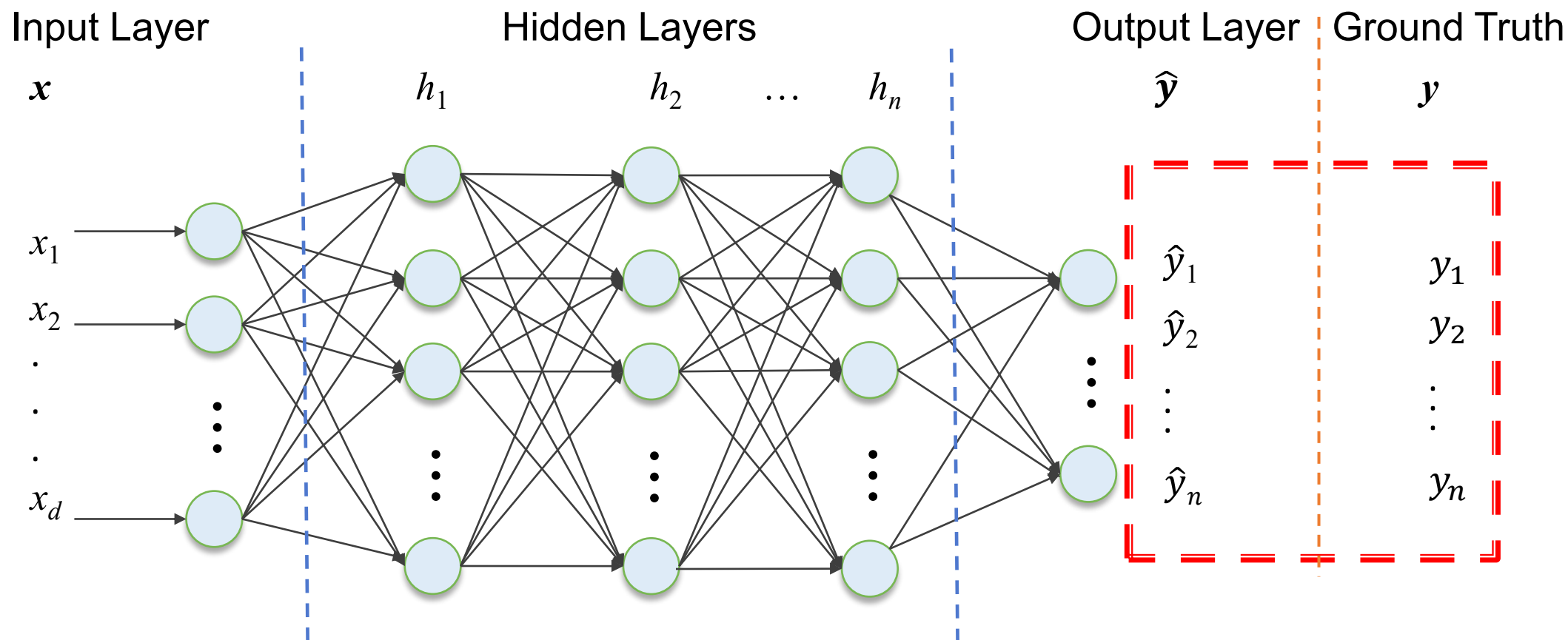
The softmax activation function is designed to work with multi-class classification tasks, where an input needs to be assigned to one of several classes. The function is used to transform raw, unbounded scores (which are often referred to as logits) into a probability distribution over multiple classes.

$$p(y = i) = \frac{e^{z_i}}{\sum_{j=1}^J e^{z_j}} \quad \text{for } i = 1, 2, \dots, J$$

Maxout: $\max_j(z_j)$

Loss Function

to measure how close our predictions are to the ground truth label.



Regression Problem

MSE: Mean Squared Error
均方誤差

$$MSE(\hat{\mathbf{y}} - \mathbf{y}) = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2$$

MAE: Mean Absolute Error
平均絕對值誤差

$$MAE(\hat{\mathbf{y}} - \mathbf{y}) = \frac{1}{n} \sum_{i=1}^n |\hat{y}_i - y_i|$$

Classification Problem

Cross-Entropy
交叉熵

$$H(\hat{\mathbf{y}} - \mathbf{y}) = - \sum_{i=1}^n y_i \log(\hat{y}_i)$$

Entropy 熵

Information of a random event a with probability $p(a)$

$$I(a) = \log \frac{1}{p(a)} = -\log(p(a))$$

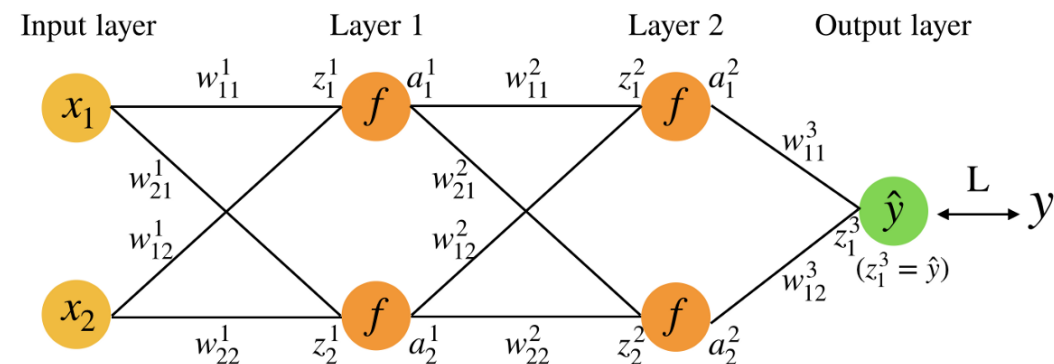
Events $\{a_1, a_2, \dots, a_J\}$ with probabilities $\{p(a_1), p(a_2), \dots, p(a_J)\}$

Entropy: the average information per source output

$$H = -\sum_{j=1}^J p(a_j) \log(a_j)$$

Backpropagation 反向傳播

3Blue1Brown-Backpagation



w_{11}^1 ← Layer 1
← Layer 1 Neuron 1 to Input layer Neuron 1

$z_1^1 = x_1 w_{11}^1 + x_2 w_{12}^1$, z_1^1 : activation function input

$a_1^1 = f(z_1^1)$, f : activation function, a_1^1 : activation output

$\hat{y}$: predict value

<https://brilliant.org/wiki/backpropagation/>

- Backpropagation, short for "backward propagation of errors," is an algorithm for supervised learning of artificial neural networks using gradient descent.
- Given an artificial neural network and an error function, the method calculates the gradient of the error function with respect to the neural network's weights. It is a generalization of the delta rule for perceptrons to multilayer feedforward neural networks.

Deep Learning Networks

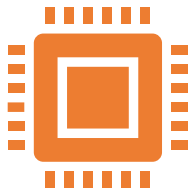
深度學習神經網路架構 - 各種不同的演算法

- Fully Connected Network (FCN) 全連接網絡
- Convolution Neural Network (CNN) 卷積神經網絡
- Recurrent Neural Network (RNN) 循環神經網絡
- Generative Adversarial Network (GAN) 生成對抗網路
- AutoEncoder(AE) 自動編碼器
- Transformer 轉換模型
- Vision Transformer (ViT) 視覺轉換模型
- Diffusion Model 擴散模型

類別	模型名稱	核心特色	主要應用
CNN 系列	ResNet, DenseNet, EfficientNet	Skip connection, Dense connection, scaling	影像辨識
RNN 系列	LSTM, GRU, Transformer-XL	長序列記憶	語音 / 文本
生成模型	GAN, VAE, Diffusion Models	生成逼真樣本	影像合成
注意力模型	Transformer, ViT, Swin	全域關聯建模	NLP, Vision
自監督學習	SimCLR, BYOL, MAE	無標籤特徵學習	表徵學習
圖神經網路 (GNN)	GCN, GraphSAGE, GAT	圖結構學習	社交網路、推薦系統
擴散模型 (Diffusion)	DDPM, Stable Diffusion	噪聲反向生成	影像生成、修復
混合架構	CNN + Transformer (ConvNeXt, CoAtNet)	卷積與注意力融合	高效影像理解

Deep Learning Frameworks

深度學習框架



TensorFlow

Developed by Google, released in 2015

Widely used in industry; strong production deployment tools (e.g., TFLite, TensorFlow Serving)



Keras

Originally independent, later integrated into TensorFlow

Very simple and beginner-friendly; great for rapid prototyping



Pytorch

Developed by Meta (formerly Facebook), released in 2016

Dynamic computation graph; highly flexible; extremely popular in research

Learning path

學習路徑

或許開始時許多程式片段看不懂 不要急，
功夫一步一步練！



<https://youtu.be/yquHVgpMxcs>

Run the sample program to check the
result

執行範例程式檢視結果

Understand the meaning and usage of
each command by tracing the code line
by line

追蹤程式碼，查詢並了解每個指令意
義與用法

Lab1 MLP MNIST Classification

建立專案

成功執行看結果感受即可
程式架構細節以後再來學習

```
import torch
from torch import nn
from torch.utils.data import DataLoader
from torchvision import datasets
from torchvision.transforms import ToTensor
import time
```

```
# Download training data from open datasets.
training_data = datasets.FashionMNIST(
    root="data",
    train=True,
    download=True,
    transform=ToTensor(),
)
```

```
# Download test data from open datasets.
test_data = datasets.FashionMNIST(
    root="data",
    train=False,
    download=True,
    transform=ToTensor(),
)
```

```
batch_size = 64
```

Lab2 CNN MNIST Classification

建立專案

成功執行看結果感受即可
程式架構細節以後再來學習

```

import torch
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim
import torchvision
import torchvision.transforms as transforms
import numpy as np
import matplotlib.pyplot as plt
import time

```

```

device = (
    "cuda"
    if torch.cuda.is_available()
    else "cpu"
)
print(f"Using {device} device")

```

```

batch_size = 16
trainset = torchvision.datasets.MNIST(root=
trainloader = torch.utils.data.DataLoader(tr

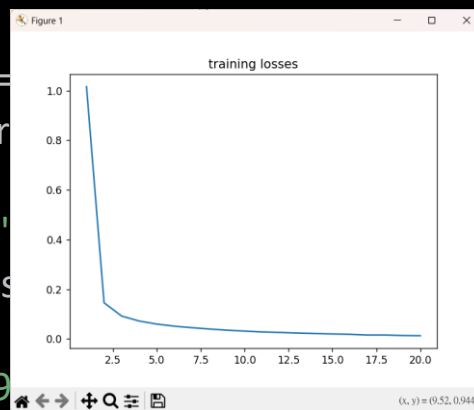
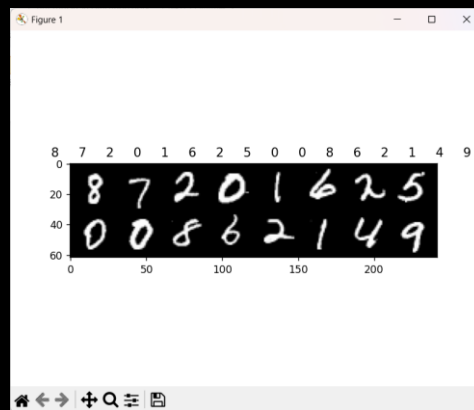
testset = torchvision.datasets.MNIST(root=
testloader = torch.utils.data.DataLoader(test

```

```

classes = ('0', '1', '2', '3', '4', '5', '6', '7', '8', '9')
# get some random training images
dataiter = iter(trainloader)
images, labels = next(dataiter)

```



```

C:\Users\user\anaconda3\envs\torch5800\python.exe
E:\Pytorch\L001Lab2mnistClassidfication\main.py
Using cuda device
Net(
  (conv1): Conv2d(1, 6, kernel_size=(5, 5), stride=(1, 1))
  (pool): MaxPool2d(kernel_size=2, stride=2, padding=0,
dilation=1, ceil_mode=False)
  (conv2): Conv2d(6, 16, kernel_size=(5, 5), stride=(1, 1))
  (fc1): Linear(in_features=256, out_features=120, bias=True)
  (fc2): Linear(in_features=120, out_features=84, bias=True)
  (fc3): Linear(in_features=84, out_features=10, bias=True)
)
epoch: 1 loss:1.0157 time:12.822022914886475
epoch: 2 loss:0.1454 time:23.93037509918213
epoch: 3 loss:0.0921 time:35.15130090713501
epoch: 4 loss:0.0721 time:46.46295166015625
epoch: 5 loss:0.0598 time:57.65580868721008
epoch: 6 loss:0.0513 time:68.88234901428223
epoch: 7 loss:0.0453 time:80.07895350456238
epoch: 8 loss:0.0399 time:91.39352560043335
epoch: 9 loss:0.0352 time:102.8779125213623
epoch: 10 loss:0.0316 time:114.17871904373169
epoch: 11 loss:0.0282 time:125.45399618148804
epoch: 12 loss:0.0261 time:136.99183416366577
epoch: 13 loss:0.0237 time:148.3039608001709

```